

UNIVERSIDADE ESTADUAL DO PIAUÍ
CIÊNCIA DA COMPUTAÇÃO

Relatório Técnico

**CampusCoin: Criação de um Token ERC-20 na Blockchain
Ethereum**

Disciplina: Blockchain, Smart Contracts e Web3

Tema: Desenvolvimento e implantação de um token digital original

Tecnologias: Solidity, OpenZeppelin, Ethereum/EVM, Remix IDE, MetaMask,
ethers.js

Autor(a): Juliana Sued da Silva Oliveira

TERESINA

2026

SUMÁRIO

1	RELATÓRIO TÉCNICO	2
1.1	Introdução	2
1.2	Dados do contrato	2
1.3	Estrutura do contrato	2
1.4	Funcionalidades implementadas	3
1.5	Tokenomics	3
1.6	Custos (Gas)	3
1.7	Publicação na blockchain	4
1.8	Evidências da implementação	4
1.9	Interface Web3	5
1.10	Testes realizados	6
1.11	Conclusão	6

1 RELATÓRIO TÉCNICO

1.1 Introdução

Este relatório apresenta o desenvolvimento de um contrato inteligente implementado em Solidity e publicado em uma rede compatível com a Ethereum. O objetivo do projeto é demonstrar, na prática, o uso de *smart contracts*, abordando sua estrutura, principais funcionalidades, forma de implantação e consulta pública por meio do Etherscan.

O contrato foi desenvolvido com foco em clareza, organização e funcionamento verificável. A proposta atende aos requisitos do enunciado, incluindo a criação do contrato, sua publicação na blockchain, a identificação do endereço gerado e a apresentação das funcionalidades implementadas.

1.2 Dados do contrato

A Tabela 1 apresenta as principais informações do contrato inteligente desenvolvido.

Tabela 1 – Dados do contrato inteligente

Item	Informação
Nome do contrato	CampusCoin (CAMP)
Linguagem utilizada	Solidity
Rede utilizada	Ethereum Sepolia
Endereço do contrato	0x4D8fee345cc1482456008B26453282D6cb0E17a
Link do Etherscan	< https://sepolia.etherscan.io/address/0x4D8fee345cc1482456008B26453282D6cb0E17a >
Ferramenta de desenvolvimento	Remix IDE
Carteira utilizada	MetaMask

1.3 Estrutura do contrato

O contrato CampusCoin foi desenvolvido sobre a biblioteca **OpenZeppelin**, herdando os módulos ERC20, ERC20Burnable e Ownable. Trata-se de um token digital do padrão ERC-20, com nome **CampusCoin**, símbolo **CAMP** e 18 casas decimais.

A estrutura principal do contrato é composta por:

- um mapping(address => bool) chamado blacklist, que armazena os endereços bloqueados;
- um constructor que emite o supply inicial para a carteira administradora;

- funções públicas para emissão (mint) e bloqueio de endereços (definirBlacklist), restritas ao dono via onlyOwner;
- validações com require para impedir operações inválidas;
- as funções padrão do ERC-20 herdadas da OpenZeppelin, como transfer, balanceOf e totalSupply.

1.4 Funcionalidades implementadas

As funcionalidades implementadas no contrato foram:

1. **Transferência e consulta de saldo:** funções padrão do ERC-20 (transfer, balanceOf, totalSupply) que permitem enviar tokens e consultar saldos e supply.
2. **Mint (emissão):** cria novos tokens, aumentando o supply; restrita ao dono do contrato (onlyOwner).
3. **Burn (queima):** destrói tokens do próprio saldo, reduzindo o supply em circulação.
4. **Blacklist (bloqueio):** permite ao dono bloquear endereços de enviar ou receber tokens, usando mapping e require.
5. **Controle de acesso:** o padrão Ownable garante que apenas o dono execute funções administrativas.

Foram implementadas três funcionalidades além do padrão básico (Mint, Burn e Blacklist), superando o mínimo de duas exigido no enunciado.

1.5 Tokenomics

A CampusCoin possui supply inicial de **1.000.000 CAMP**, emitidos no momento da publicação para a carteira administradora. O supply é dinâmico, podendo aumentar via mint e diminuir via burn.

O token foi concebido com propósito acadêmico, podendo ser utilizado em eventos, serviços da biblioteca e premiações de alunos. A distribuição parte de 100% do supply na carteira administradora, de onde os tokens circulam por transferências e novas emissões controladas.

1.6 Custos (Gas)

A publicação foi realizada em testnet, com custos pagos em SepoliaETH, sem valor real. O deploy do contrato consumiu aproximadamente **1.279.400 de gas**, equivalente a cerca de

0,0042 SepoliaETH. As operações de transferência, mint e burn possuem custo significativamente menor, por não publicarem novo bytecode na rede.

1.7 Publicação na blockchain

Após o desenvolvimento, o contrato foi compilado e publicado na rede Ethereum Sepolia. A publicação foi realizada por meio do Remix IDE, utilizando a carteira MetaMask para assinar a transação.

Depois da implantação, a rede gerou um endereço único para o contrato. Esse endereço permite localizar o contrato na blockchain e interagir com ele por meio de ferramentas como o Etherscan.

O endereço do contrato publicado é:

0x4D8f6eea345cc1482456008B26453282D6cb0E17a

O contrato também pode ser consultado publicamente no Etherscan pelo seguinte link:

<https://sepolia.etherscan.io/address/0x4D8f6eea345cc1482456008B26453282D6cb0E17a>

1.8 Evidências da implementação

Nesta seção são apresentadas imagens que comprovam o desenvolvimento, a publicação e a consulta do contrato inteligente na blockchain.

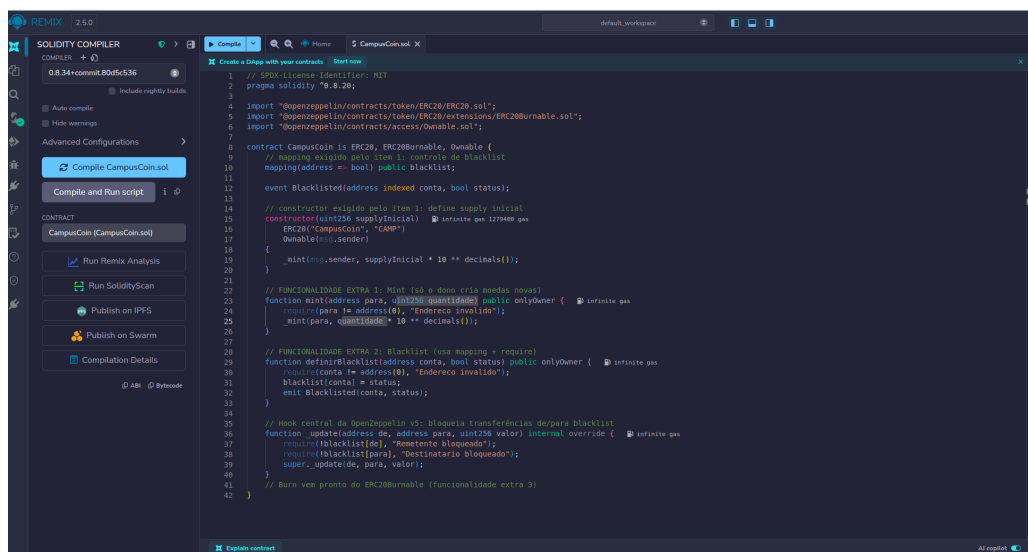


Figura 1 – Código do contrato inteligente no Remix IDE

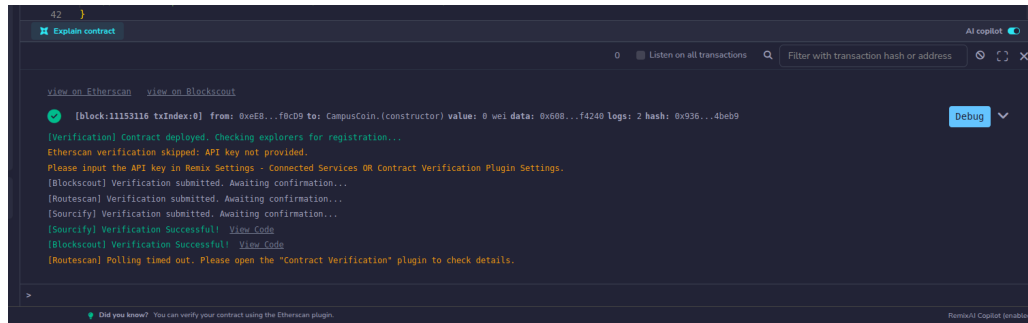


Figura 2 – Deploy do contrato inteligente realizado no Remix IDE

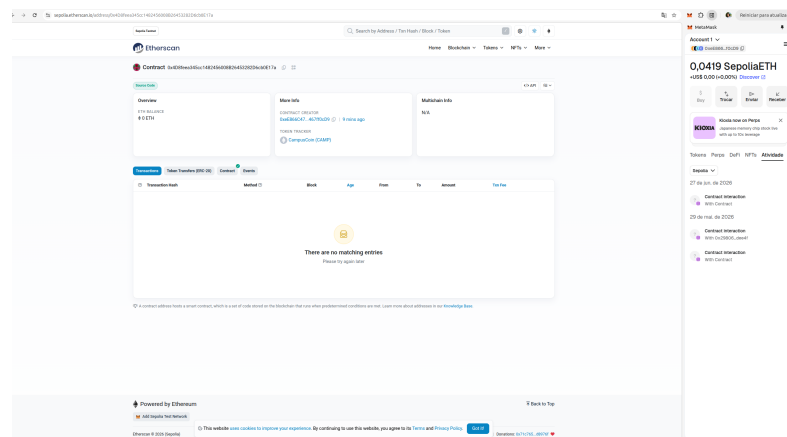


Figura 3 – Contrato publicado e consultado no Etherscan

1.9 Interface Web3

Foi desenvolvida uma interface web utilizando HTML, JavaScript e a biblioteca `ethers.js`, que se comunica com o contrato na rede Sepolia. A interface permite conectar a carteira MetaMask, consultar o saldo de tokens e realizar transferências diretamente do navegador, conforme a Figura 4.

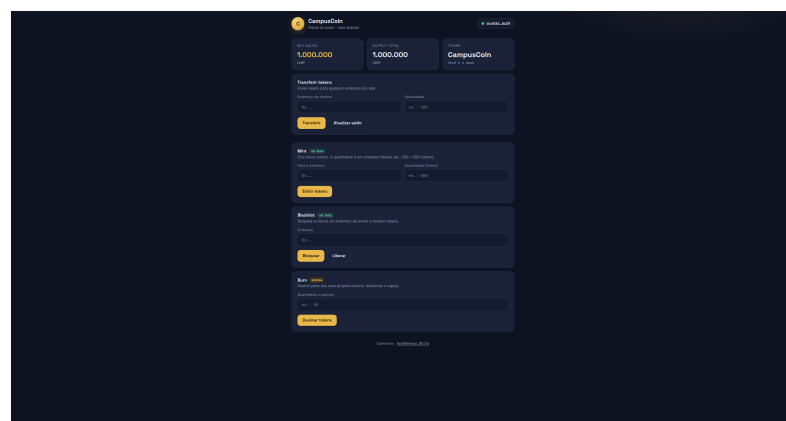


Figura 4 – Interface Web3 conectada à MetaMask exibindo saldo e transferência

1.10 Testes realizados

Foram realizados testes básicos para verificar se o contrato funcionava conforme esperado. Os principais testes estão descritos na Tabela 2.

Tabela 2 – Testes realizados no contrato

Teste	Descrição	Resultado
Compilação	Verificação do código Solidity no Remix	Sucesso
Deploy	Publicação na rede Ethereum Sepolia	Sucesso
Consulta de saldo	Execução de <code>balanceOf</code>	Sucesso
Transferência	Execução de <code>transfer</code> entre contas	Sucesso
Mint	Emissão de novos tokens pelo dono	Sucesso
Burn	Queima de tokens do próprio saldo	Sucesso
Blacklist	Transferência para endereço bloqueado	Revertida (esperado)
Controle de acesso	Mint executado por conta não-dona	Revertida (esperado)
Verificação pública	Contrato verificado no Etherscan/Sourcify	Sucesso

Os testes confirmaram que o contrato foi publicado corretamente e que suas principais funcionalidades puderam ser executadas conforme o esperado.

1.11 Conclusão

O desenvolvimento da CampusCoin permitiu aplicar, na prática, conceitos de blockchain, Solidity, padrão ERC-20 e publicação de aplicações descentralizadas. O token foi implementado com a biblioteca OpenZeppelin, incluindo as funcionalidades de mint, burn e blacklist, compilado, publicado e verificado na rede Ethereum Sepolia, e disponibilizado para consulta pública no Etherscan.

Além do contrato, foi desenvolvida uma interface Web3 que conecta a carteira MetaMask e permite consultar saldo e transferir tokens diretamente do navegador, usando a biblioteca ethers.js. Os testes confirmaram tanto o funcionamento correto das operações quanto a eficácia das proteções de segurança. Como evolução futura, o token poderia incorporar mecanismos de staking ou governança estudantil.